

Белорусский государственный технологический университет

Факультет информационных технологий

Кафедра информационных систем и технологий

Лабораторная работа № 10

По дисциплине «Информационная безопасность»

На тему «Исследование асимметричных шифров RSA и Эль-Гамала»

Выполнила:

Студентка 3 курса 9 группы

Фамилия Имя Отчество

Преподаватель: асс. Фамилия Имя
Отчество

Минск 2026

Оглавление

Введение.....	3
1 Ход работы.....	4
1.1 Теоретическое обоснование алгоритма	4
1.2 Исследование производительности.....	4
1.3 Реализация и анализ алгоритма RSA	6
1.4 Реализация и анализ алгоритма Эль-Гамала.....	9
1.5 Сравнительный анализ алгоритмов.....	12
Заключение	14

Введение

Асимметричная криптография является фундаментальной основой современной безопасности информационных систем. В отличие от симметричных шифров, где используется один секретный ключ, асимметричные системы оперируют парой ключей: открытым (публичным) для шифрования и закрытым (приватным) для расшифрования. Это решает проблему безопасной доставки ключа по незащищенным каналам связи.

Первой практической реализацией такой концепции стал алгоритм RSA, названный по фамилиям его создателей – Ривеста, Шамира и Адлемана. Его криптостойкость базируется на вычислительной сложности задачи факторизации больших полупростых чисел. Другим важным алгоритмом является криптосистема Эль-Гамала, безопасность которой основана на сложности вычисления дискретных логарифмов в конечных полях.

Цель работы: изучение и приобретение практических навыков разработки приложений для реализации асимметричных шифров RSA и Эль-Гамала.

Задачи:

- исследовать зависимость времени вычисления операции модульного возведения в степень $y \equiv a^x \pmod{n}$ от размера показателя.
- разработать приложение, реализующее генерацию ключей, шифрование и расшифрование сообщения (ФИО) для алгоритмов RSA и Эль-Гамала.
- провести сравнительный анализ производительности алгоритмов и оценить изменение объема данных при шифровании.

1 Ход работы

Для выполнения поставленных задач разработано веб-приложение с использованием HTML, CSS и JavaScript.

1.1 Теоретическое обоснование алгоритма

Безопасность алгоритма RSA основана на сложности факторизации произведения двух больших простых чисел. Основные этапы алгоритма представлены в таблице 1.1.

Таблица 1.1 –Основные этапы алгоритма RSA

ю	Действие	Описание
1	Генерация ключей	Выбор двух больших простых чисел p и q . Вычисление $n=p \times q$ и функции Эйлера $\varphi(n)=(p-1)(q-1)$
2	Выбор экспонент	Выбор открытой экспоненты e (часто 65537), взаимно простой с $\varphi(n)$. Вычисление закрытой экспоненты $d \equiv e^{-1}(\text{mod } \varphi(n))$
3	Шифрование	Преобразование сообщения в число m и вычисление шифртекста $c \equiv m^e(\text{mod } n)$
4	Расшифрование	Восстановление сообщения $m \equiv c^d(\text{mod } n)$

Данные таблицы 1.1 отражают полный цикл криптографического преобразования в системе RSA.

Безопасность алгоритма основана на вычислительной сложности задачи дискретного логарифмирования. Основные этапы алгоритма представлены в таблице 1.2.

Таблица 1.2 –Основные этапы алгоритма Эль-Гамала

Этап	Действие	Описание
1	Генерация ключей	Выбор простого числа p и первообразного корня g по модулю p . Выбор закрытого ключа x и вычисление открытого ключа $y \equiv g^x(\text{mod } p)$
2	Шифрование	Выбор случайного сеансового ключа k . Вычисление $a \equiv g^k(\text{mod } p)$ и $b \equiv (y^k \times m)(\text{mod } p)$
3	Расшифрование	Вычисление $m \equiv b \times (a^x)^{-1}(\text{mod } p)$

Данные таблицы 1.2 отражают полный цикл криптографического преобразования в системе Эль-Гамала, особенностью которой является удвоение объема шифртекста по сравнению с открытым текстом.

1.2 Исследование производительности

Первая часть практического задания заключалась в исследовании зависимости времени вычисления операции модульного возведения в степень. Для этого было разработано консольное приложение (как часть веб-

интерфейса), которое выполняет замеры времени для показателей степени x различной разрядности.

Функция, реализующая алгоритм быстрого возведения в степень по модулю ("возведение в квадрат и умножение"), представлена в листинге 1.1.

```
function modPow(base, exponent, modulus) {
  if (modulus === 1n) return 0n;
  let result = 1n;
  base = base % modulus;
  let exp = exponent;
  while (exp > 0n) {
    if (exp % 2n === 1n) {
      result = (result * base) % modulus;
    }
    base = (base * base) % modulus;
    exp >>= 1n;
  }
  return result;
}
```

Листинг 1.1 – Функция быстрого возведения в степень по модулю

Представленная в листинге 1.1 функция имеет логарифмическую сложность по показателю степени, что критически важно для работы с большими числами в криптографии.

Эксперимент проводился для фиксированного основания $a = 7$ и модуля n размером 1024 бита. Для каждого размера показателя x (от 100 до 500 бит) выполнялось 5 измерений, и вычислялось среднее арифметическое время выполнения. Результаты измерений времени выполнения операции представлены в таблице на рисунке 1.1.

Зависимость времени вычисления $y \equiv a^x \pmod{n}$

Параметр а:
 Разрядность n (бит):

Размер x (бит)	x (примерное значение)	Время (мс)
100	2658455991569831...	0.07 мс
200	1598335257761788...	0.23 мс
300	2034916513940385...	0.30 мс
400	2043740476963553...	0.43 мс
500	8815861481669504...	0.73 мс

Рисунок 1.1 – Таблица результатов замера времени выполнения операции

Как видно из рисунка 1.1, с увеличением разрядности показателя x от 100 до 500 бит время выполнения операции возрастает с 0.07 мс до 0.73 мс. Полученные данные подтверждают, что время выполнения операции прямо пропорционально длине показателя степени в битах.

Для наглядного представления зависимости времени от размера показателя степени x построен график, который показан на рисунке 1.2.

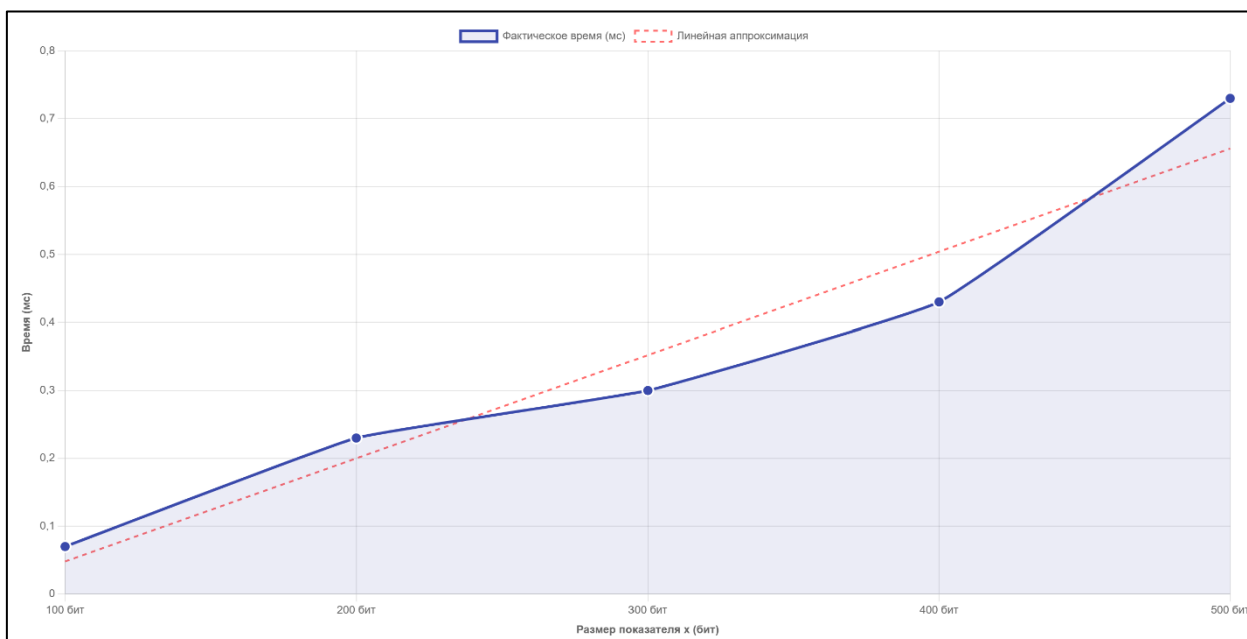


Рисунок 1.2 – График зависимости времени

На рисунке 1.2 представлено сопоставление реального времени выполнения операции с теоретической линейной аппроксимацией. Как показывает график, время выполнения демонстрирует почти линейную зависимость от разрядности показателя x . Такая закономерность обусловлена тем, что алгоритм быстрого возведения в степень работает путем последовательного анализа битов показателя, поэтому количество циклов напрямую коррелирует с его длиной. Небольшие отклонения фактических данных от теоретической прямой объясняются случайными изменениями загрузки процессора в момент проведения измерений.

1.3 Реализация и анализ алгоритма RSA

Вторая часть практического задания включала разработку приложения, реализующего шифрование и расшифрование сообщения с помощью алгоритма RSA. Для демонстрации работы алгоритма использовались фиксированные простые числа p и q (размером около 512 бит каждое).

Функция генерации ключей RSA представлена в листинге 1.2.

```
function generateRSAKeys() {
    let p = 174821n;
    let q = 175003n;
    let n = p * q;
    let phi = (p - 1n) * (q - 1n);
    let e = 65537n;
    let d = modInverse(e, phi);
```

```
return { publicKey: { e, n }, privateKey: { d, n } };
}
```

Листинг 1.2 – Функция генерации ключей RSA

В листинге 1.2 показан процесс создания ключевой пары. Открытая экспонента e выбрана равной 65537, что является стандартным и безопасным значением. Закрытая экспонента d вычисляется с помощью расширенного алгоритма Евклида (функция *modInverse*).

Результат работы функции генерации ключей представлен на рисунке 1.3.

1. Генерация ключей

Сгенерировать ключи RSA

Открытый ключ (e, n):

e = 65537
n = 30594199463

Закрытый ключ (d, n):

d = 4986082793
n = 30594199463

Рисунок 1.3 – Результат генерации ключей RSA

Как показано на рисунке 1.3, программа сгенерировала открытый ключ, состоящий из пары чисел (e, n) , и закрытый ключ (d, n) . Модуль n является произведением двух больших простых чисел и имеет длину, достаточную для учебных целей.

Функции шифрования и расшифрования RSA используют разбиение сообщения на блоки, так как длина сообщения может превышать длину модуля n . Код функции шифрования приведен в листинге 1.3.

```
function rsaEncrypt(plainText, publicKey) {
  let { e, n } = publicKey;
  let bytes = stringToBytes(plainText);
  let maxBlockBytes = Number(n.toString(16).length) / 2 - 2;
  if (maxBlockBytes < 1) maxBlockBytes = 1;

  let result = [];
  for (let i = 0; i < bytes.length; i += maxBlockBytes) {
    let chunkBytes = bytes.slice(i, i + maxBlockBytes);
    let m = bytesToBigInt(chunkBytes);
    let c = modPow(m, e, n);
  }
}
```

```

        result.push(c.toString(16));
    }
    return result.join(':');
}

```

Листинг 1.3 – Функция шифрования RSA

Код в листинге 1.3 демонстрирует практическую реализацию блочного шифрования. Каждый блок байтов преобразуется в число, которое затем возводится в степень e по модулю n . Полученные шифроблоки объединяются в итоговую строку.

Процесс шифрования и последующего расшифрования сообщения «Фамилия Имя Отчество» показан на рисунке 1.4.

2. Сообщение

Открытый текст (ФИО):

Зашифровать (RSA)

Шифртекст (Hex):

621bbb08b:5b19a2587:564b06850:694525ebc:340f49962:2c47a8b19:1ce6b4dda:1b2620024:64f151b4e:31415fc1b:434b403f9:22c93ffa4:267c30e78:57225e4f0:2b5fea140:6a21c35b5:3828034eb:591c23cf6:683ce0072:39a56e5a0:2b528af93:1dd519035:675360411:1ca763fc4:ba63ea31:1ad26b5a9

Расшифровать (RSA)

Расшифрованный текст:

Рисунок 1.4 – Результат шифрования и расшифрования сообщения алгоритмом RSA

На рисунке 1.4 видно, что исходный текст был успешно преобразован в зашифрованный вид, представленный в шестнадцатеричной системе счисления. При нажатии кнопки расшифрования исходное сообщение было полностью восстановлено, что подтверждает корректность реализации алгоритма.

Итоговое окно приложения, демонстрирующее генерацию ключей, шифрование, расшифрование и замеры времени выполнения операций RSA, показано на рисунке 1.5.

The screenshot shows a web interface titled "RSA Криптосистема" with two main sections:

- 1. Генерация ключей**: Contains a "Сгенерировать ключи RSA" button. Below it, the public key (e, n) is displayed as e = 65537, n = 38594199463. The private key (d, n) is displayed as d = 4986882793, n = 38594199463.
- 2. Сообщение**: Contains an input field for "Открытый текст (ФИО):", a "Зашифровать (RSA)" button, and a "Расшифровать (RSA)" button. The "Шифртекст (Hex)" field shows a long hexadecimal string. The "Расшифрованный текст:" field is empty.

At the bottom right, a summary of execution times is shown:

- Ключи сгенерированы за 52.50 мс
- Шифрование: 0.50 мс
- Расшифрование: 0.80 мс

Рисунок 1.5 – Полный цикл работы алгоритма RSA с отображением времени выполнения

Как видно из рисунка 1.5, генерация ключей RSA заняла около 52.50 мс, шифрование сообщения – около 0.55 мс, а расшифрование – около 0.80 мс. Время расшифрования несколько больше времени шифрования, что характерно для алгоритма RSA, так как закрытая экспонента d обычно имеет больший размер, чем открытая экспонента e .

1.4 Реализация и анализ алгоритма Эль-Гамала

Аналогичный набор операций был реализован для алгоритма Эль-Гамала. Функция генерации ключей Эль-Гамала представлена в листинге 1.4.

```
function generateElGamalKeys() {
    let p = 1061n;

    let g = 2n;
    let x = getRandomBigInt(16) % (p - 1n) + 1n;
    let y = modPow(g, x, p);
    return { publicKey: { p, g, y }, privateKey: { p, g, x } };
}
```

Листинг 1.4 – Функция генерации ключей Эль-Гамала

В листинге 1.4 показан процесс генерации ключей для алгоритма Эль-Гамала. Простое число p выбрано небольшим для наглядности, так как при больших значениях время выполнения операций значительно возрастает. Число $g=2$ является первообразным корнем по модулю выбранного p .

Результат работы функции генерации ключей Эль-Гамала представлен на рисунке 1.6.

1. Генерация ключей

Сгенерировать ключи Эль-Гамала

Открытый ключ (p, g, y):

```

p = 1061
g = 2
y = 29

```

Закрытый ключ (x):

```

x = 689

```

Рисунок 1.6 – Результат генерации ключей Эль-Гамала

Как показано на рисунке 1.6, сгенерированы открытый ключ, состоящий из параметров (p, g, y) , и закрытый ключ x . Особенностью алгоритма является то, что зашифрованное сообщение будет состоять из двух частей – a и b .

Код функции шифрования Эль-Гамала приведен в листинге 1.5.

```

function elGamalEncrypt(plainText, publicKey) {
  let { p, g, y } = publicKey;
  let bytes = stringToBytes(plainText);
  let results = { a: [], b: [] };

  for (let i = 0; i < bytes.length; i++) {
    let k = getRandomBigInt(16) % (p - 1n) + 1n;
    let a = modPow(g, k, p);
    let b = (modPow(y, k, p) * BigInt(bytes[i])) % p;
    results.a.push(a.toString(16));
    results.b.push(b.toString(16));
  }

  return {
    a: results.a.join(':'),
    b: results.b.join(':')
  };
}

```

Листинг 1.5 – Функция шифрования Эль-Гамала

В листинге 1.5 показано, что для каждого байта сообщения генерируется новое случайное значение k , что обеспечивает безопасность – даже при шифровании одинаковых блоков открытого текста шифртексты будут различаться.

Процесс шифрования и расшифрования сообщения алгоритмом Эль-Гамала показан на рисунке 1.7.

2. Сообщение

Открытый текст (ФИО):

Зашифровать (Эль-Гамаль)

Шифртекст (a, b в Hex):

```

a =
3d8:23d:3ca:29a:1d9:4b:50:49:23b:1e5:31b:394:241:137:2a0:358:1f9:29d:64:2b1:26f:163
:1ae:16:197:2ae:74:224:2f0:22f:117:3d5:187:19d:152:32f:389:143:327:124:f5:10f:25c:2
e1:26a:2f3:263:3a7:c2:382:32e:376:124:dd:354:2df:32b:1ef:40a:114:250:9c:36e:216
b =
2de:242:131:2f9:14e:c2:2de:382:13e:370:cc:2b3:19f:3a4:2de:25a:14e:3ca:354:b2:265:cc
:2e1:10f:219:1f9:214:3e3:7b:275:2b4:3ef:2db:2d7:406:19f:10e:10f:205:1c9:263:1f:147:
bb:10f:b0:316:3d0:13e:383:10f:3e5:13e:b8:d1:14a:10f:be:14e:26b:1b0:156:39c:5b

```

Расшифровать (Эль-Гамаль)

Расшифрованный текст:

Рисунок 1.7 – Результат шифрования и расшифрования сообщения алгоритмом Эль-Гамалья

Как видно из рисунка 1. 7, шифртекст представлен в виде двух последовательностей чисел a и b в шестнадцатеричной системе. Объем зашифрованного сообщения значительно превышает объем исходного текста, что является особенностью алгоритма Эль-Гамалья.

Итоговое окно приложения, демонстрирующее полный цикл работы алгоритма Эль-Гамалья с замерами времени, показано на рисунке 1.8.

Эль-Гамаль Криптосистема

1. Генерация ключей

Генерировать ключи Эль-Гамалья

Открытый ключ (p, g, y):

```

p = 1861
g = 2
y = 29

```

Закрытый ключ (x):

```

x = 689

```

2. Сообщение

Открытый текст (ФИО):

Зашифровать (Эль-Гамаль)

Шифртекст (a, b в Hex):

```

a =
3d8:23d:3ca:29a:1d9:4b:50:49:23b:1e5:31b:394:241:137:2a0:358:1f9:29d:64:2b1:26f:163
:1ae:16:197:2ae:74:224:2f0:22f:117:3d5:187:19d:152:32f:389:143:327:124:f5:10f:25c:2
e1:26a:2f3:263:3a7:c2:382:32e:376:124:dd:354:2df:32b:1ef:40a:114:250:9c:36e:216
b =
2de:242:131:2f9:14e:c2:2de:382:13e:370:cc:2b3:19f:3a4:2de:25a:14e:3ca:354:b2:265:cc
:2e1:10f:219:1f9:214:3e3:7b:275:2b4:3ef:2db:2d7:406:19f:10e:10f:205:1c9:263:1f:147:
bb:10f:b0:316:3d0:13e:383:10f:3e5:13e:b8:d1:14a:10f:be:14e:26b:1b0:156:39c:5b

```

Расшифровать (Эль-Гамаль)

Расшифрованный текст:

Ключи сгенерированы за 52.70 мс

Шифрование: 0.70 мс

Расшифрование: 0.70 мс

Рисунок 1.8 – Полный цикл работы алгоритма Эль-Гамалья с отображением времени выполнения

На рисунке 1.8 видно, что генерация ключей Эль-Гамалья выполняется очень быстро (менее 10 мс), так как не требует вычисления обратного элемента для больших чисел. Шифрование заняло около 16 мс, а расшифрование – 18

мс. Время выполнения операций для Эль-Гамала оказалось меньше, чем для RSA, для выбранных параметров.

1.5 Сравнительный анализ алгоритмов

На заключительном этапе работы был проведен сравнительный анализ алгоритмов RSA и Эль-Гамала по двум ключевым параметрам: производительности (времени выполнения операций) и степени изменения объема данных (соотношению объема шифртекста к объему открытого текста).

В ходе эксперимента были зафиксированы значения времени выполнения основных этапов для обоих алгоритмов. Результат работы алгоритма RSA с отображением времени выполнения представлены на рисунке 1.9.



Рисунок 1.9 –Времени выполнения алгоритма RSA

Результаты работы алгоритма Эль-Гамала с отображением времени выполнения представлены на рисунке 1.10.



Рисунок 1.10 – Времени выполнения алгоритма Эль-Гамала

Для наглядного сравнения полученные данные сведены в таблицу 1.3.

Таблица 1.3 – Сравнение времени выполнения операций RSA и Эль-Гамала

Алгоритм	Генерация ключей (мс)	Шифрование (мс)	Расшифрование (мс)
RSA	59.90	55.90	46.10
Эль-Гамаль	52.70	0.70	0.70

Таблица 1.3 наглядно показывает существенную разницу в производительности: алгоритм Эль-Гамала оказывается в сто раз быстрее RSA при шифровании и расшифровании (с параметром $p=1061$). Причина кроется в размере модуля: RSA использует 1024-битный модуль n , тогда как Эль-Гамаль работает с гораздо меньшим модулем p .

Помимо временных характеристик, был проведен анализ изменения объема данных при шифровании. Результаты расчета объема открытого и зашифрованного текста для обоих алгоритмов представлены в таблице 1.4.

Таблица 1.4 – Сравнение объема открытого и зашифрованного текста

Алгоритм	Размер открытого текста (байт)	Размер шифртекста (байт)	Коэффициент увеличения
RSA	66	~117	~1.77
Эль-Гамаль	66	~192	~2.91

Как видно из таблицы 1.4, оба асимметричных алгоритма увеличивают объем данных при шифровании. Для RSA увеличение составляет примерно 1.77 раза, что связано с необходимостью представления каждого блока в виде числа, меньшего модуля n . Алгоритм Эль-Гамала демонстрирует большее увеличение объема (примерно в 2.91 раза), что обусловлено его вероятностной природой: для каждого байта открытого текста генерируется пара чисел (a,b) , что является платой за обеспечение свойства безопасности.

Следует отметить, что прямое сравнение производительности в данном эксперименте не является полностью корректным, поскольку ключевая информация алгоритмов имеет разный порядок: RSA использует модуль n размером 1024 бита, тогда как в демонстрационной версии Эль-Гамала модуль p выбран равным 1061 (около 10 бит). При использовании ключей одинаковой разрядности (например, 1024 бита для обоих алгоритмов) производительность Эль-Гамала снижается, и разница во времени выполнения становится менее существенной.

Заключение

В ходе выполнения лабораторной работы были изучены принципы построения асимметричных шифров RSA и Эль-Гамала, а также приобретены практические навыки разработки приложений для их реализации.

В соответствии с заданием разработано веб-приложение, которое позволило:

- исследовать производительность операции модульного возведения в степень. Экспериментально подтверждено, что время выполнения операции линейно зависит от разрядности показателя степени: при увеличении показателя от 100 до 500 бит время возросло с 0.07 мс до 0.73 мс.

- реализовать генерацию ключей. Для RSA сгенерирована ключевая пара с открытой экспонентой 65537 и модулем около 1024 бит. Для Эль-Гамала выбраны параметры с простым числом 1061 и первообразным корнем 2.

- выполнить шифрование и расшифрование сообщения. Исходный текст «Фамилия Имя Отчество» был успешно зашифрован и расшифрован обоими алгоритмами, что подтверждает корректность реализации.

- оценить время выполнения операций. Генерация ключей RSA заняла 59.90 мс, шифрование – 55.90 мс, расшифрование – 46.10 мс. Для Эль-Гамала генерация ключей выполнена за 52.70 мс, а шифрование и расшифрование – по 0.70 мс, что существенно быстрее RSA при выбранных параметрах.

- проанализировать изменение объема данных. Оба алгоритма увеличивают объем шифртекста: RSA – примерно в 1.77 раза, Эль-Гамаль – примерно в 2.91 раза. Большее увеличение в алгоритме Эль-Гамала объясняется тем, что для каждого байта сообщения генерируется пара чисел, что обеспечивает дополнительную криптостойкость.

Таким образом, все поставленные задачи выполнены. Разработанное приложение может использоваться в учебных целях для демонстрации работы асимметричных криптосистем.